

自然语言处理-实验一

Si-Cheng Feng¹

¹ Nankai University

摘要

本文关注自然语言处理领域的情感分类任务，针对 IMDB 数据集分别采用 CNN 和 LSTM 模型分别实现了较好的分类效果。

1. 实验环境与准备工作

实验平台与环境. 本次实验平台与环境如表 1

数据集. IMDB (Internet Movie Database) 数据集是一个广泛用于自然语言处理任务的情感分类数据集，主要包含来自 IMDB 网站的电影评论。该数据集的核心特点如下：

1. 数据量: 包含 50,000 条高评分和低评分的电影评论，平均每条评论约有 240 个单词。
2. 任务: 该数据集主要用于文本分类任务，目标是根据评论的内容判断情感（正面或负面）。
3. 数据分割: 数据集分为训练集和测试集，每部分包含 25,000 条评论，且正负评论数量均衡。
4. 应用场景: 常用于情感分析、文本分类、机器学习和深度学习模型的训练与评估。

2. 实验原理

原理. 情感分类任务是自然语言处理 (Natural Language Processing, NLP) 中的一个重要任务，旨在识别和分类文本中表达的情感倾向。通常，这些情感倾向可以分为正面、负面和中性，或者根据需求划分为更细致的类别（如高兴、愤怒、悲伤等）。情感分类在多个实际场景中得到了广泛应用，包括但不限于以下领域：

1. 产品评价分析: 分析用户对产品的评价，判断市场反应。
2. 社交媒体监测: 监控用户在社交平台上的情绪变化，分析社会舆情。
3. 品牌声誉管理: 通过分析用户评论或新闻报道中的情感倾向，帮助企业管理品牌形象。

任务本质. 将文本依据其蕴含的情感分成不同类别，通常使用的类别包括正面、负面和中性。为完成此任务，通常采用机器学习或深度学习技术，如卷积神经网络 (CNN)、长短时记忆网络 (LSTM) 以及预训练的语言模型 (BERT) 等。

在本次实验中，我们首先采用 CNN 模型然后再利用 LSTM 模型进行文本情感分类。

模型. 下面我们对本次实验涉及的两个模型进行细致的介绍：

1. CNN. 卷积神经网络 (CNN) 是深度学习中的一种特殊类型神经网络，主要用于处理具有网格结构的数据，最典型的应用是在图像数据的处理上。CNN 在图像分类、目标检测、图像分割等任务中取得了巨大的成功。与传统神经网络不同，CNN 的结构能够有效捕捉数据中的局部特征，并通过参数共享大大减少了模型的复杂度。其结构如图 1

- **卷积层 (Convolutional Layer):** 卷积层是 CNN 的核心层，主要作用是通过卷积核 (filter) 对输入数据进行局部感受。卷积操作可以提取数据的局部特征，例如图像中的边缘、纹理等。每个卷积核在数据的局部区域内进行滑动，生成特征图 (feature

表 1. 实验环境说明表

VS Code 版本	Conda 版本	Python 版本	CPU	GPU
1.94.2	24.1.2	3.11.7	Intel(R) Xeon(R) Gold 6430	单卡 RTX4090

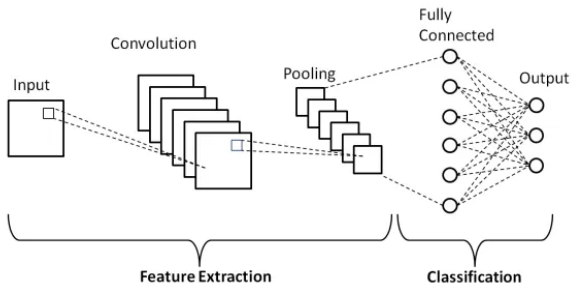


图 1. CNN

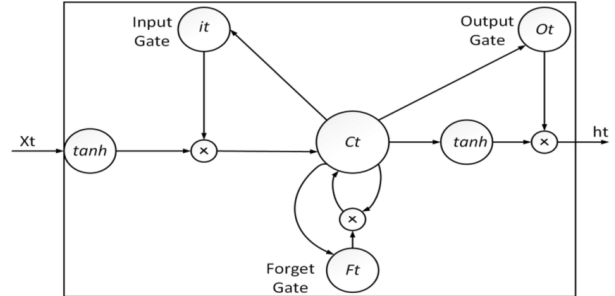


图 2. LSTM

map)。通过多个卷积层的叠加，CNN 可以提取从低级特征到高级特征的信息。

- **激活函数 (Activation Function):** 通常在卷积层之后添加非线性激活函数，例如 ReLU (Rectified Linear Unit)，用于引入非线性能力，帮助模型学习复杂的模式。
- **池化层 (Pooling Layer):** 池化层用于下采样特征图，减少数据的维度并降低计算复杂度。常见的池化方式包括最大池化 (Max Pooling) 和平均池化 (Average Pooling)。池化层可以帮助模型具有一定的平移不变性。
- **全连接层 (Fully Connected Layer):** 在卷积层和池化层之后，通常会加入一到两层全连接层。全连接层将特征图展平为一维向量，并与输出类别进行映射。最后一层全连接层的输出维度等于任务中的类别数。
- **输出层 (Output Layer):** 对于分类任务，输出层通常使用 Softmax 函数，输出每个类别的概率分布。

基本工作流程: 输入数据首先经过若干卷积层提取特征，接着通过池化层进行特征下采样，随后通过全连接层进行分类或回归等任务。CNN 通过这种局部连接、参数共享和多层特征提取的方式，能够在处理高维数据时具有较强的鲁棒性和计算效率。

2. LSTM. 长短时记忆网络 (LSTM) [1]是一种特殊的循环神经网络 (Recurrent Neural Network, RNN)，主要用于处理和预测基于时间序列的数据。与传统的 RNN 不同，LSTM 通过设计专门的记忆单元，能够有效地解决 RNN 的长期依赖问题，防止梯度消失或爆炸，从而在较长时间跨度内捕获序列信息。LSTM 的结构由多个循环单元组成，每个单元包含以下主要部分，如图 2

- **记忆单元 (Memory Cell):** LSTM 的核心是记忆单元，用于存储网络随时间步长传递的信息。通过对记忆单元的控制，LSTM 可以决定哪些信息需要保留，哪些信息需要丢弃。
- **遗忘门 (Forget Gate):** 遗忘门用于决定哪些历史信息需要丢弃。其输出是介于 0 和 1 之间的值，值越接近 0，表示遗忘的信息越多，值越接近 1，表示保留的信息越多。计算公式为：

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

其中， f_t 是遗忘门的输出， W_f 是权重矩阵， h_{t-1} 是前一个时刻的隐藏状态， x_t 是当前时刻的输入， σ 表示 Sigmoid 函数。

- **输入门 (Input Gate):** 输入门决定当前时刻的输入对记忆单元的更新量。其输出同

样通过 Sigmoid 函数计算：

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

- **候选记忆更新** (Candidate Memory Update): 候选记忆通过 $\tanh$ 函数进行计算, 生成新的候选状态信息, 准备更新记忆单元:

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

- **输出门** (Output Gate): 输出门控制记忆单元的输出, 通过结合当前的隐藏状态和输入计算下一时刻的隐藏状态:

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$$

- **记忆单元更新** (Memory Cell Update): 记忆单元状态 C_t 通过结合遗忘门和输入门的输出进行更新:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

- **隐藏状态更新** (Hidden State Update): 隐藏状态 h_t 是输出门与记忆单元的组输出:

$$h_t = o_t \cdot \tanh(C_t)$$

这种结构使得 LSTM 能够在长时间序列中保留关键信息, 而忽略无关的信息, 从而在自然语言处理、时间序列预测等任务中表现优异。

3. 编程实现与分析

核心代码构成分为三部分:

1. **数据预处理**. 数据预处理是整个情感分类任务的关键步骤之一。使用 `torchtext` 库来加载 IMDB 数据集并进行标记化和数值化处理。

```
# 加载IMDB数据集
```

```
train_iter, test_iter = IMDB(split=('train', 'test'))
```

```
# 使用basic_english标记器
```

```
tokenizer = get_tokenizer("basic_english")
```

```
# 构建词汇表
```

```
def yield_tokens(data_iter):
```

```
    for __, text in data_iter:
```

```
        yield tokenizer(text)
```

```
vocab = build_vocab_from_iterator(
```

```
    yield_tokens(train_iter),
```

```
    specials=["<unk>", "<pad>"])
```

```
vocab.set_default_index(vocab["<unk>"])
```

```
# 数值化处理示例
```

```
text_pipeline = lambda x: vocab(tokenizer(x))
```

```
# 核心示例, 数值化某个文本
```

```
example_text = "This movie is fantastic!"
```

```
numericalized_text = text_pipeline(example_text)
```

- **数据加载**: 使用 IMDB 数据集的训练和测试集。
- **标记化**: 使用 `get_tokenizer` 定义了 `basic_english` 作为标记器。
- **词汇表构建**: 使用 `build_vocab_from_iterator` 从训练数据中提取词汇, 并设置特殊标记 "<unk>" 和 "<pad>"。
- **数值化处理**: 将输入文本转化为对应的词汇表索引序列。

2. **模型**. 这是最为核心的代码, 用于构建完成目标任务的模型架构。

3. **训练与评估**. 这部分主要通过两个函数分别实现:

- **Train** 函数用于训练模型。首先, 将模型设置为训练模式, 这会启用 `dropout` 等训练时特有的操作。然后, 初始化两个空列表 `epoch_losses` 和 `epoch_accs`, 用于存储每个批次的损失和准确率。遍历数据加载器中的每个批次。对于每个批次, 将输入数据和标签移动到指定的设备 (如 GPU), 然

后通过模型进行预测。计算预测结果的损失和准确率，并将它们添加到相应的列表中。在进行反向传播和优化步骤之前，先将优化器的梯度清零，然后反向传播损失，最后更新模型参数。最后，返回所有批次损失和准确率的平均值。

- Evaluate 函数用于评估模型的性能。首先，将模型设置为评估模式，这会关闭 dropout 等训练时特有的操作。然后，初始化两个空列表 epoch_losses 和 epoch_accs，用于存储每个批次的损失和准确率。在不计算梯度的上下文中，遍历数据加载器中的每个批次。对于每个批次，将输入数据和标签移动到指定的设备（如 GPU），然后通过模型进行预测。计算预测结果的损失和准确率，并将它们添加到相应的列表中。最后，返回所有批次损失和准确率的平均值。

4. 实验结果

4.1. CNN

这里经过调参后最后确定的超参数如下：

1. -max_length: 256
2. -min_freq: 5
3. -embedding_dim: 300
4. -n_filters: 100
5. -filter_sizes: [3, 5, 7]
6. -dropout_rate: 0.25
7. -n_epochs: 10

最终的模型效果为 84.9%，Train 阶段模型在训练和验证集上的准确率和损失的曲线如图 3。

1. 训练集损失随着训练轮数的增加而逐步降低，这表明模型在训练集上正在逐渐学习，并且损失在不断减少。验证集损失同样呈现下降趋势，但在训练到一定程度后（约在第 6 轮左右），损失趋于稳定，甚至有很轻微上升的迹象。这种情况表明模型在验证集上可能出现了**过拟合**，即模型在训练集上表现很好，但在验证集上不能继续提升。

2. 训练集准确率随着轮数增加而不断上升，说明模型在训练集上的表现持续提高。验证集准确率在最初的几轮训练后迅速提高，并在约第 6 轮达到稳定值，最终稳定在约 84.9%。这表明模型在验证集上有较好的泛化能力，但同样存在过拟合的可能性，因为在后期训练中，验证集的准确率提升不大。

4.2. LSTM

这里经过调参后最后确定的超参数如下：

1. -hidden_size: 256
2. -min_freq: 5
3. -num_layers: 5
4. -output_size: 5
5. -dropout_rate: 0.25
6. -n_epochs: 20

最终的模型效果为 83.3%，Train 阶段模型在训练和验证集上的准确率和损失的曲线如图 4。

1. 训练集损失随着训练轮数的增加，损失持续下降，表明模型在训练数据上学习得很好，损失不断减少。验证集损失在初始几轮内，验证集的损失与训练集同步下降，但在大约第 6-8 轮左右，验证集损失开始有所波动，且最终趋于稳定，甚至出现轻微上升。这可能表明模型开始出现过拟合，即在训练集上的表现持续优化，但在验证集上的泛化能力未能继续提升。
2. 训练集准确率随着训练轮数的增加，准确率显著上升，表现非常好，接近 90%。验证集准确率验证集的准确率在前几轮有较快的提升，但在大约第 8 轮后开始趋于平缓，最终在 83% 时达到稳定。这表明模型在验证集上的泛化能力不错，但也显示出一定的瓶颈。
3. 过拟合现象：从验证集损失的轻微上升和准确率趋于平缓可以看出，模型在训练集上表现很好，但在验证集上并未能继续提高，这可能是过拟合的表现。可以考虑：增加正则化，增大 dropout 的比例，以减少过拟合；减少训练轮数，使用早停训练策略可能会更有效。

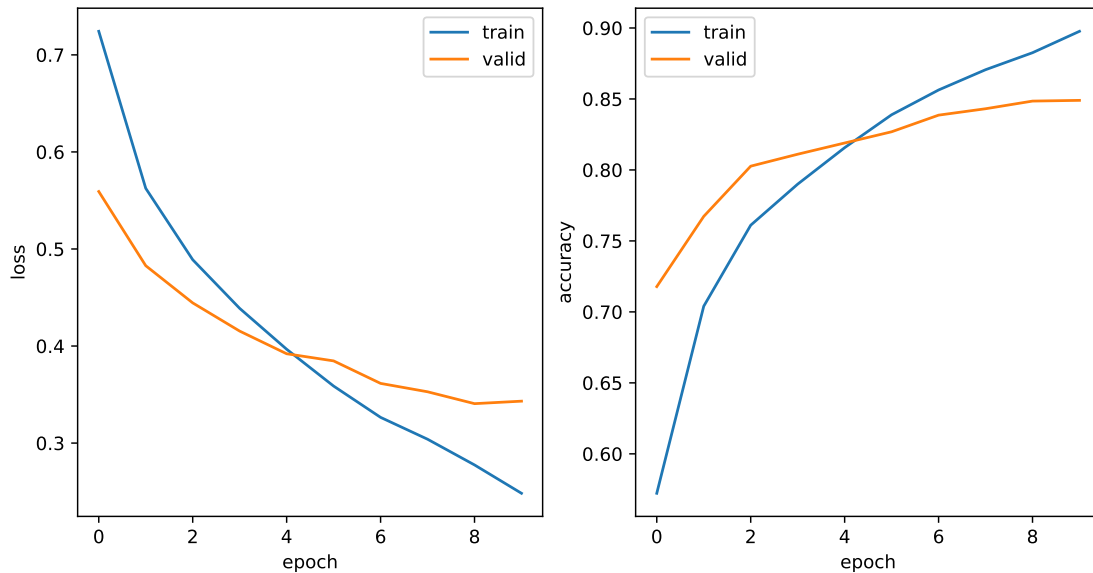


图 3. CNN-train, valid 中 acc 和 loss 曲线

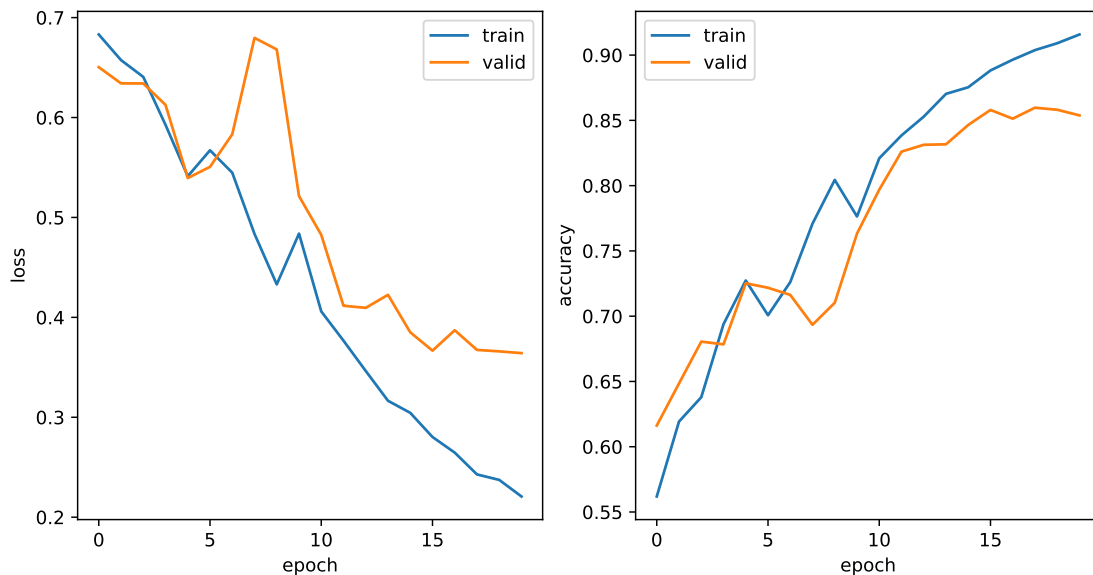


图 4. LSTM-train, valid 中 acc 和 loss 曲线

5. 总结与展望

在这次实验中，我成功完成了所有要求的任务，包括调整超参数并基于框架完善了 textCNN 模型，用于文本情感分类。此外，我还设计了基于 LSTM

的模型来提升效果，并对测试数据进行了评估。三次实验在测试集上的准确率均超过了 80%。通过这个实验，我不仅学到了很多新知识，也为今后的研究奠定了坚实的基础。

参考文献

- [1] A. Graves and A. Graves. Long short-term memory. Supervised sequence labelling with recurrent neural networks, pages 37–45, 2012. [2](#)